Viernes 10/04 — Variables, Tipos de Datos y Conversión entre Tipos

---

1. ¿Qué es una variable?

Una **variable** es un nombre que le damos a un espacio en la memoria del ordenador donde guardamos un dato. Piensa en ella como una etiqueta pegada a una caja: la etiqueta es el nombre y dentro de la caja está el valor.

```
edad = 30
nombre = "Laura"
temperatura = 22.5
esta_lloviendo = False
```

En el ejemplo anterior hemos creado cuatro "cajas" (variables), cada una con un valor distinto dentro.

Asignación con `=`

El símbolo `=` en Python no significa "es igual a" (eso se escribe `==`). Significa **"guarda este valor en esta variable"**:

```
# Lee: "la variable precio RECIBE el valor 9.99"
precio = 9.99

# Podemos cambiar el valor en cualquier momento
precio = 12.50  # Ahora precio vale 12.50
```

Reglas para nombrar variables

Regla	Ejemplo válido	Ejemplo inválido
Solo letras, números y guiones bajos	`edad_usuario`	`edad-usuario`
No puede empezar por un número	`precio1`	`1precio`
No puede ser una palabra reservada	`mi_clase`	`class`
Sensible a mayúsculas/minúsculas	`Nombre ≠ nombre`	—

Palabras reservadas son las que Python usa internamente: `if`, `else`, `for`, `while`, `class`, `def`, `True`, `False`, `None`, etc. No puedes usarlas como nombres de variables.

Asignación múltiple

Python permite asignar varias variables en una sola línea:

```
# Asignación múltiple
nombre, edad, ciudad = "Carlos", 28, "Sevilla"

# Asignar el mismo valor a varias variables
x = y = z = 0
```

---

2. Tipos de datos fundamentales

Cada valor en Python pertenece a un **tipo de dato**. El tipo determina qué operaciones puedes hacer con ese valor.

2.1 Enteros (`int`)

Números sin decimales. Pueden ser positivos, negativos o cero.

```
edad = 25
temperatura_minima = -3
habitantes = 1000000
saldo = 0
```

2.2 Decimales (`float`)

Números con punto decimal. En Python se usa punto, no coma.

```
precio = 19.99
pi = 3.14159
porcentaje = 0.15
temperatura = -2.5
```

> **Atención**: En España escribimos `19,99` pero Python usa el punto: `19.99`.

2.3 Cadenas de texto (`str`)

Texto encerrado entre comillas simples (`'...'`) o dobles (`"..."`). Ambas son equivalentes.

```
nombre = "Ana López"
direccion = 'Calle Gran Vía, 10'
codigo_postal = "29001"  # ¡Es texto, no un número!
mensaje = "Dijo: 'hola'"  # Comillas dentro de comillas
```

¿Cuándo un número es texto? Cuando no necesitas hacer cálculos matemáticos con él. Los códigos postales, números de teléfono y DNIs son texto, aunque contengan dígitos.

2.4 Booleanos (`bool`)

Solo pueden tener dos valores: `True` (verdadero) o `False` (falso). Se usan para representar condiciones.

```
es_mayor_de_edad = True
tiene_permiso = False
esta_activo = True
```

> **Nota**: `True` y `False` van siempre con la primera letra en mayúscula.

2.5 NoneType (`None`)

Representa la "ausencia de valor". Es útil cuando una variable existe pero aún no tiene un dato asignado.

```
resultado = None  # Todavía no tenemos el resultado
```

---

3. Conocer el tipo de un dato: `type()`

La función `type()` nos dice de qué tipo es un valor o variable:

```
print(type(42))          # <class 'int'>
print(type(3.14))        # <class 'float'>
print(type("Hola"))      # <class 'str'>
print(type(True))        # <class 'bool'>
print(type(None))        # <class 'NoneType'>

edad = 30
print(type(edad))        # <class 'int'>
```

Esto es especialmente útil para **depurar** (encontrar errores). Cuando algo no funciona como esperas, comprueba el tipo de tus variables.

---

4. Conversión entre tipos (casting)

Muchas veces necesitamos convertir un dato de un tipo a otro. Esto se llama **casting** o conversión de tipo.

4.1 Texto a número: `int()` y `float()`

Recuerda que `input()` siempre devuelve texto. Si el usuario escribe un número, necesitas convertirlo:

```
# Sin conversión: esto concatena textos, no suma
edad_texto = "25"
print(edad_texto + "5")  # Resultado: "255" (¡no es lo que queremos!)
```

```
# Con conversión: ahora sí es una suma numérica
edad_numero = int("25")
print(edad_numero + 5)   # Resultado: 30

# Para decimales, usamos float()
precio = float("19.99")
print(precio * 2)         # Resultado: 39.98
```

4.2 Número a texto: `str()`

A veces necesitas convertir un número a texto para concatenarlo con otras cadenas:

```
edad = 30
# Esto da error:
# print("Tengo " + edad + " años")  # TypeError

# Solución: convertir a texto
print("Tengo " + str(edad) + " años")  # Tengo 30 años

# Alternativa más cómoda: usar comas en print()
print("Tengo", edad, "años")  # Tengo 30 años
```

4.3 Entero a decimal y viceversa

```
# Entero a decimal
numero = 10
decimal = float(numero)
print(decimal)       # 10.0

# Decimal a entero (se pierden los decimales, NO redondea)
precio = 19.99
entero = int(precio)
print(entero)        # 19 (no 20)

# Para redondear, usa round()
redondeado = round(19.99)
print(redondeado)    # 20
```

4.4 A booleano: `bool()`

Casi cualquier cosa se puede convertir a booleano. La regla general es:

- **Falso**: `0`, `0.0`, `""` (texto vacío), `None`, `False`
- **Verdadero**: todo lo demás

```
print(bool(0))       # False
print(bool(1))       # True
```

```
print(bool(-5))       # True
print(bool(""))       # False
print(bool("Hola"))   # True
print(bool(None))     # False
```

4.5 Tabla resumen de conversiones

Origen	Destino	Función	Ejemplo	Resultado
`str`	`int`	`int()`	`int("42")`	`42`
`str`	`float`	`float()`	`float("3.14")`	`3.14`
`int`	`str`	`str()`	`str(100)`	`"100"`
`float`	`str`	`str()`	`str(9.99)`	`"9.99"`
`int`	`float`	`float()`	`float(10)`	`10.0`
`float`	`int`	`int()`	`int(9.99)`	`9`
Cualquiera	`bool`	`bool()`	`bool(0)` / `bool("Sí")`	`False` / `True`

---

5. Operaciones básicas con cada tipo

Con números (`int` y `float`)

```
a = 15
b = 4

print(a + b)     # Suma: 19
print(a - b)     # Resta: 11
print(a * b)     # Multiplicación: 60
print(a / b)     # División: 3.75 (siempre devuelve float)
print(a // b)    # División entera: 3 (descarta los decimales)
print(a % b)     # Módulo (resto): 3
print(a ** b)    # Potencia: 50625 (15 elevado a 4)
```

Con texto (`str`)

```
nombre = "Ana"
apellido = "López"

# Concatenar (unir textos)
nombre_completo = nombre + " " + apellido
print(nombre_completo)  # Ana López

# Repetir texto
linea = "-" * 30
```

```
print(linea)  # ------------------------------

# Longitud de una cadena
print(len(nombre_completo))  # 10
```

Con booleanos (`bool`)

```
# Los booleanos se pueden usar con operadores lógicos
es_adulto = True
tiene_carnet = False

print(es_adulto and tiene_carnet)  # False (ambos deben ser True)
print(es_adulto or tiene_carnet)   # True (al menos uno es True)
print(not tiene_carnet)            # True (invierte el valor)
```

---

6. Formateo de salida con f-strings

Los **f-strings** (cadenas formateadas) son la forma más cómoda de mezclar texto y variables en Python. Se escriben poniendo una `f` antes de las comillas:

```
nombre = "Carlos"
edad = 35
salario = 1850.50

# Sin f-string (menos legible)
print("Me llamo " + nombre + " y tengo " + str(edad) + " años")

# Con f-string (mucho más limpio)
print(f"Me llamo {nombre} y tengo {edad} años")
print(f"Mi salario es {salario:.2f} €")  # Con 2 decimales: 1850.50 €
```

Formateo de números en f-strings

```
precio = 1234.5
porcentaje = 0.156

print(f"Precio: {precio:.2f} €")       # Precio: 1234.50 €
print(f"Porcentaje: {porcentaje:.1%}")  # Porcentaje: 15.6%
print(f"Cantidad: {precio:,.2f}")       # Cantidad: 1,234.50
```

---

7. Programas de ejemplo combinando todo

Ejemplo 1: Ficha de contacto

```python
# Recogemos datos del usuario
print("=== NUEVA FICHA DE CONTACTO ===")
nombre = input("Nombre: ")
telefono = input("Teléfono: ")
email = input("Email: ")

# Mostramos la ficha formateada
print()
print(f"{'='*30}")
print(f"Nombre:    {nombre}")
print(f"Teléfono:  {telefono}")
print(f"Email:     {email}")
print(f"{'='*30}")
```

Ejemplo 2: Conversor de temperatura

```python
# Convertir grados Celsius a Fahrenheit
celsius_texto = input("Introduce la temperatura en °C: ")
celsius = float(celsius_texto)

fahrenheit = (celsius * 9/5) + 32

print(f"{celsius:.1f} °C equivalen a {fahrenheit:.1f} °F")
```

Ejemplo 3: Cálculo de nómina básica

```python
# Cálculo simplificado de nómina mensual
print("=== CÁLCULO DE NÓMINA ===")
nombre = input("Nombre del empleado: ")
salario_bruto = float(input("Salario bruto mensual (€): "))
retencion_irpf = float(input("Retención IRPF (%): "))

# Cálculos
importe_irpf = salario_bruto * (retencion_irpf / 100)
seguridad_social = salario_bruto * 0.0635  # 6.35% aproximado
salario_neto = salario_bruto - importe_irpf - seguridad_social

# Resultado
print()
print(f"--- Nómina de {nombre} ---")
print(f"Salario bruto:       {salario_bruto:>10.2f} €")
print(f"IRPF ({retencion_irpf}%):        -{importe_irpf:>10.2f} €")
print(f"Seg. Social (6.35%): -{seguridad_social:>10.2f} €")
print(f"{'—'*35}")
print(f"Salario neto:        {salario_neto:>10.2f} €")
```

Resumen de la sesión

Concepto	Idea clave
Variable	Nombre que apunta a un valor almacenado en memoria
`int`	Números enteros: `42`, `-3`, `0`
`float`	Números decimales: `3.14`, `-2.5`
`str`	Texto entre comillas: `"Hola"`
`bool`	Verdadero (`True`) o falso (`False`)
`None`	Ausencia de valor
`type()`	Devuelve el tipo de un dato
Casting	Convertir entre tipos: `int()`, `float()`, `str()`, `bool()`
f-strings	Formato moderno para mezclar texto y variables

Siguiente clase: Lunes 13/04 — Operadores aritméticos, lógicos y construcción de expresiones.